Race Condition (Lost Update) on Shared Records via Silent Data Overwriting

Severity: Medium

CVSS Score: 6.1 (Medium)

CWE

- CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization (Race Condition)
- CWE-664: Improper Control of a Resource Through its Lifetime
- CWE-841: Improper Enforcement of Behavioral Workflow

Affected Asset

- URL: https://crm-stage.crusource.com
- Parameter/Storage Key: Contact/Entity update endpoints (e.g., PUT /api/contacts/{id} or similar)
- Environment: Staging
- Date Discovered: 13-10-2026

Why this is a vulnerability:

- When two users load the same entity (e.g., a contact record) and edit different fields simultaneously, each user's browser holds a copy of the record from when the page was loaded. When they save, the browser submits the entire record — not just the fields the user modified.
- If User A and User B both load a contact, User A updates the phone number, and User B updates the job title, the user who saves second submits the entire record with the older values from when their page was loaded. The server accepts the write unconditionally, overwriting the other user's change.
- The result is **silent data loss**. Neither user is warned that their change conflicted with another user's edit. The application does not detect the conflict, does not ask the user to resolve it, and does not preserve both independent changes.
- This indicates the application lacks **optimistic concurrency control** (such as a RowVersion / ETag / updated_at check) or **pessimistic locking** at the database level. Without one of these controls, concurrent updates are processed sequentially and the last write always wins.

Delivery vector (how an attacker would exploit it):

- **Silent Data Loss:** Any two users editing the same record within the same window causes one user's change to be overwritten without warning.
- **Data Integrity Compromise:** Records become inconsistent — audit logs, reports, and downstream systems may reflect values the user never intended to save.
- **Business Impact in a CRM Context:** Contacts, deals, tickets, and notes are commonly edited by sales reps, support agents, and admins simultaneously. In a high-traffic CRM, this results in lost customer information, missed follow-ups, and incorrect revenue forecasts.
- **Malicious Abuse (Lower Likelihood, Higher Impact):** A malicious authenticated user who knows a target is editing a record could intentionally time a conflicting save to overwrite sensitive data (e.g., revert a phone number, revert a flag, or revert a status change).
- **Reproducibility:** The issue is deterministic with proper timing — it is not an edge case, it is a guaranteed outcome when two saves occur on the same record.

Remediation

1. **Implement optimistic concurrency control.** Include a version identifier (RowVersion, updated_at timestamp, or ETag) with every record. When the client submits an update, the server compares the client's version against the current version. If they differ, the server rejects the update with 409 Conflict and returns the current state to the client.
2. **Implement read-only control.** Implement entity-level edit locks or conflict detection to prevent concurrent modifications. When a second user attempts to edit a record currently being modified by another user, display a notification and to take a step further, block the save until the conflict is resolved.
3. **Return conflict responses to the frontend.** The UI must display the conflict to the user – show them their change, the other user's change, and let them choose how to merge. Silent rejection is not sufficient.
4. **Use conditional writes at the database layer.** For example: UPDATE contacts SET ... WHERE id = ? AND updated_at = ?. If zero rows are affected, the record was modified since it was loaded.
5. **Alternatively, use granular patch endpoints.** Instead of PUT /api/contacts/{id} (full replacement), use PATCH /api/contacts/{id} where the client submits only the fields it modified. The server merges those fields into the current record. This eliminates most lost-update scenarios for field-level edits.
6. **Add database-level row locking for critical operations** if optimistic concurrency is not feasible. Note that this trades throughput for correctness and should be used only on high-value entities.
7. **Audit existing data** to determine whether silent overwrites have already occurred in production. Compare audit logs against current record values if the CRM maintains a change history.

Summary

The application does not detect the conflict, does not return a conflict response, and does not prompt the user to resolve it. The result is silent data loss and data integrity compromise – particularly impactful in a CRM where multiple team members routinely work on the same contacts, deals, and tickets. Remediation requires implementing optimistic concurrency control (version/ETag checks) at the API layer, returning 409 Conflict responses to the frontend, and either prompting the user for merge resolution or switching to granular PATCH-style updates that preserve independent field changes.